Waveshare 6" HD e-Paper Display (IT8951)
Raspberry Pi 2B + Home Assistant Integration

Requirements
• Raspberry Pi 2B

• Waveshare 6" HD e-Paper HAT with IT8951 Driver Board

• VCOM value from the FPC ribbon cable (e.g. -2.30) — printed on the cable

• Raspberry Pi OS Trixie (Debian 13) freshly installed

• Home Assistant running somewhere on the network

• LAN cable or USB Wi-Fi adapter

Step 1 — Enable SPI
SPI is the communication interface between the Pi and the display. Enable it once:

```
sudo raspi-config
```

Interface Options → SPI → Yes → OK → Finish

```
sudo reboot
```

Step 2 — Install Dependencies
On Raspberry Pi OS Trixie (Debian 13) some package names differ from older versions. Use exactly these names:

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y git python3-pip python3-venv libopenjp2-7 libtiff-dev libatlas3-base
```

Note: libtiff5 and libatlas-base-dev no longer exist on Trixie.

Step 3 — Install BCM2835 Library
The IT8951 driver needs the BCM2835 library for GPIO access. Must be compiled manually:

```
cd ~
wget http://www.airspayce.com/mikem/bcm2835/bcm2835-1.75.tar.gz
tar zxvf bcm2835-1.75.tar.gz
cd bcm2835-1.75
./configure
make
sudo make check
sudo make install
cd ~
```

Step 4 — Compile Waveshare IT8951 C Driver
Compile once — required as a base by the Python library:

```
git clone https://github.com/waveshareteam/IT8951-ePaper.git
cd IT8951-ePaper/Raspberry
sudo make clean
sudo make -j4
cd ~
```

The warning 'EPD_IT8951_LoadImgStart defined but not used' is harmless.

Step 5 — Create Python Virtual Environment
An isolated Python environment prevents conflicts with system Python:

```
python3 -m venv ~/epaper-env
source ~/epaper-env/bin/activate
```

After activation, (epaper-env) appears before the prompt.

Step 6 — Install Python Libraries

```
pip install Pillow requests RPi.GPIO spidev
pip install git+https://github.com/GregDMeyer/IT8951.git
```

IT8951 is not available on piwheels for Pi 2B (ARM v7) — install directly from GitHub.

Step 7 — Test the Display

```
nano ~/test_display.py
```

```
from IT8951.display import AutoEPDDisplay
from IT8951 import constants
from PIL import ImageDraw, ImageFont

VCOM = -2.30  # Enter your VCOM value here!

display = AutoEPDDisplay(vcom=VCOM, spi_hz=24000000)
display.frame_buf.paste(0xFF, [0, 0, display.width, display.height])
draw = ImageDraw.Draw(display.frame_buf)
draw.text((100, 100), "Hello! Display works!", fill=0x00)
display.draw_full(constants.DisplayModes.GC16)
print("Done!")
```

```
sudo ~/epaper-env/bin/python ~/test_display.py
```

The display briefly flashes (white-black-white) — this is the normal GC16 refresh cycle.

Step 8 — Create Home Assistant Token

1. Log in to Home Assistant
2. Click your username in the bottom left
3. Click the 'Security' tab at the top
4. Scroll down to: Long-Lived Access Tokens
5. Create token, name it e.g. epaper
6. Copy the token — it is only shown once!

Important: Never share the token or send it in chats!

Step 9 — Main Display Script (epaper_ha.py)

Fetches weather data from Home Assistant and renders the display layout. Set YOUR-IP and YOUR_TOKEN!

```
nano ~/epaper_ha.py
```

```
import requests
from IT8951.display import AutoEPDDisplay
from IT8951 import constants
from PIL import Image, ImageDraw, ImageFont
from datetime import datetime

# ■■ Settings ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
VCOM     = -2.30
HA_URL   = "http://YOUR-IP:8123"
HA_TOKEN = "YOUR_TOKEN"
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

HEADERS = {
```

```
    "Authorization": f"Bearer {HA_TOKEN}",
    "Content-Type": "application/json",
}

# Translation: HA condition codes -> English labels
WEATHER_TEXT = {
    "sunny":          "Sunny",
    "clear-night":    "Clear",
    "partlycloudy":   "Partly Cloudy",
    "cloudy":         "Cloudy",
    "fog":            "Foggy",
    "rainy":          "Rainy",
    "pouring":        "Heavy Rain",
    "snowy":          "Snowy",
    "snowy-rainy":    "Sleet",
    "hail":           "Hail",
    "lightning":      "Thunderstorm",
    "lightning-rainy": "Thunder+Rain",
    "windy":          "Windy",
    "windy-variant":  "Windy",
    "exceptional":    "Extreme",
}
DAYS = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]

def wtext(condition):
    return WEATHER_TEXT.get(condition, condition)

def get_state(entity_id):
    # Fetches HA entity state (dict with 'state' and 'attributes')
    r = requests.get(f"{HA_URL}/api/states/{entity_id}",
                     headers=HEADERS, timeout=10)
    return r.json() if r.status_code == 200 else None

def get_forecast(forecast_type):
    # Fetches weather forecast from HA
    # forecast_type: 'hourly' or 'daily'
    # Each entry: condition, datetime, temperature,
    # templow (daily only), precipitation, humidity, wind_speed
    r = requests.post(
        f"{HA_URL}/api/services/weather/get_forecasts?return_response",
        headers=HEADERS,
        json={"entity_id": "weather.forecast_home",
              "type": forecast_type},
        timeout=10
    )
    if r.status_code == 200:
        return (r.json()
                .get("service_response", {})
                .get("weather.forecast_home", {})
                .get("forecast", []))
    return []

def divider(draw, y, W, MARGIN):
    # Draws a dividing line and returns the new y position
    draw.line([(MARGIN, y), (W-MARGIN, y)], fill=0x00, width=3)
    return y + 18

def main():
    weather   = get_state("weather.forecast_home")
    hourly_fc = get_forecast("hourly")[:5]
    daily_fc  = get_forecast("daily")[:5]

    # Initialise display
    # vcom: voltage for panel (from FPC cable)
    # spi_hz: 24 MHz is stable for Pi 2B
```

```
display = AutoEPDDisplay(vcom=VCOM, spi_hz=24000000)
W, H = display.width, display.height
display.frame_buf.paste(0xFF, [0, 0, W, H])  # white background
draw = ImageDraw.Draw(display.frame_buf)

try:
    BOLD = "/usr/share/fonts/truetype/dejavu/DejaVuSans-Bold.ttf"
    REG  = "/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf"
    f_time  = ImageFont.truetype(BOLD, 110)
    f_date  = ImageFont.truetype(BOLD,  42)
    f_temp  = ImageFont.truetype(BOLD,  80)
    f_cond  = ImageFont.truetype(BOLD,  44)
    f_head  = ImageFont.truetype(BOLD,  32)
    f_label = ImageFont.truetype(BOLD,  30)
    f_word  = ImageFont.truetype(BOLD,  30)
    f_num   = ImageFont.truetype(BOLD,  42)
    f_low   = ImageFont.truetype(BOLD,  36)
    f_prec  = ImageFont.truetype(BOLD,  26)
    f_foot  = ImageFont.truetype(REG,   22)
except:
    f_time = f_date = f_temp = f_cond = f_head = f_label = \
    f_word = f_num = f_low = f_prec = f_foot = \
    ImageFont.load_default()

MARGIN = 24
y = MARGIN

# Time & Date
now = datetime.now()
draw.text((MARGIN, y), now.strftime("%H:%M"),
          font=f_time, fill=0x00)
draw.text((W-MARGIN-290, y+36), now.strftime("%d.%m.%Y"),
          font=f_date, fill=0x00)
y += 120

# Current temperature & condition
if weather:
    temp = weather["attributes"].get("temperature", "?")
    cond = weather["state"]
    draw.text((MARGIN, y),    f"{temp}deg C",  font=f_temp, fill=0x00)
    draw.text((MARGIN, y+88), wtext(cond),     font=f_cond, fill=0x00)
    y += 140
else:
    draw.text((MARGIN, y), "No weather data",  font=f_cond, fill=0x00)
    y += 60

y = divider(draw, y, W, MARGIN)

# Hourly forecast
draw.text((MARGIN, y), "Next 5 Hours", font=f_head, fill=0x00)
y += 38
col_w = (W - 2*MARGIN) // 5

has_prec = any((fc.get("precipitation") or 0) > 0
               for fc in hourly_fc)
hourly_h = 148 if has_prec else 118

if hourly_fc:
    for i, fc in enumerate(hourly_fc):
        x    = MARGIN + i*col_w
        dt   = datetime.fromisoformat(fc["datetime"]).astimezone()
        cond = fc.get("condition", "?")
        temp = fc.get("temperature", "?")
        prec = fc.get("precipitation", 0) or 0
        draw.text((x, y),      dt.strftime("%H:%M"), font=f_label, fill=0x00)
```

```
            draw.text((x, y+36),   wtext(cond),        font=f_word,  fill=0x00)
            draw.text((x, y+70),   f"{temp}deg",       font=f_num,   fill=0x00)
            if prec > 0:
                draw.text((x, y+116), f"{prec}mm",     font=f_prec,  fill=0x00)
        else:
            draw.text((MARGIN, y), "No hourly data",   font=f_word,  fill=0x00)

        y += hourly_h
        y = divider(draw, y, W, MARGIN)

        # 5-day forecast
        draw.text((MARGIN, y), "5-Day Forecast", font=f_head, fill=0x00)
        y += 38

        if daily_fc:
            for i, fc in enumerate(daily_fc):
                x    = MARGIN + i*col_w
                dt   = datetime.fromisoformat(fc["datetime"]).astimezone()
                cond  = fc.get("condition", "?")
                t_max = fc.get("temperature", "?")
                t_min = fc.get("templow", "?")
                draw.text((x, y),      DAYS[dt.weekday()], font=f_label, fill=0x00)
                draw.text((x, y+36),   wtext(cond),       font=f_word,  fill=0x00)
                draw.text((x, y+70),   f"{t_max}deg",     font=f_num,   fill=0x00)
                draw.text((x, y+116),  f"{t_min}deg",     font=f_low,   fill=0x00)
        else:
            draw.text((MARGIN, y), "No daily data",    font=f_word,  fill=0x00)

        y += 160

        # Footer
        draw.text((MARGIN, H-28),
            f"Updated: {now.strftime('%H:%M:%S')}",
            font=f_foot, fill=0x55)

        # GC16 = best quality greyscale refresh
        display.draw_full(constants.DisplayModes.GC16)
        print(f"Done at {now.strftime('%H:%M:%S')}")

    if __name__ == "__main__":
        main()
```

Step 10 — Automatic Refresh Loop (epaper_loop.py)

Runs continuously. Daytime (sun above horizon): every minute. Nighttime: every 30 minutes.

```
nano ~/epaper_loop.py
```

```
import time
import subprocess
from datetime import datetime
import requests

HA_URL   = "http://YOUR-IP:8123"
HA_TOKEN = "YOUR_TOKEN"

HEADERS = {
    "Authorization": f"Bearer {HA_TOKEN}",
    "Content-Type": "application/json",
}

def is_daytime():
    # Asks HA whether the sun is above the horizon
    try:
        r = requests.get(f"{HA_URL}/api/states/sun.sun",
                         headers=HEADERS, timeout=10)
```

```
        if r.status_code == 200:
            return r.json().get("state") == "above_horizon"
    except:
        pass
    return 7 <= datetime.now().hour < 22  # fallback

def refresh_display():
    result = subprocess.run(
        ["/home/user/epaper-env/bin/python",
         "/home/user/epaper_ha.py"],
        capture_output=True, text=True
    )
    ts = datetime.now().strftime("%H:%M:%S")
    if result.returncode == 0:
        print(f"[{ts}] Display updated")
    else:
        print(f"[{ts}] Error: {result.stderr[-200:]}")

def main():
    print("Display loop started...")
    while True:
        try:
            refresh_display()
            wait = 60 if is_daytime() else 30*60
            print(f"Next update in {wait//60} minute(s)")
            time.sleep(wait)
        except Exception as e:
            print(f"Error: {e}")
            time.sleep(60)

if __name__ == "__main__":
    main()
```

Step 11 — Autostart with Systemd

Starts automatically on boot and restarts on crash:

```
sudo nano /etc/systemd/system/epaper.service
```

```
[Unit]
Description=E-Paper Display Refresh
After=network.target

[Service]
ExecStart=/home/johannes/epaper-env/bin/python /home/user/epaper_loop.py
WorkingDirectory=/home/user
StandardOutput=journal
StandardError=journal
Restart=always
RestartSec=10
User=root

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable epaper.service
sudo systemctl start epaper.service
```

Useful commands:

```
sudo systemctl status epaper.service      # Show status
sudo journalctl -u epaper.service -f      # Live logs
sudo systemctl restart epaper.service     # Restart
sudo systemctl stop epaper.service        # Stop
```

Common Errors & Solutions

Error	Cause	Solution
bcm2835.h: No such file	BCM2835 missing	Complete Step 3
Unable to locate libtiff5	Wrong package name on Trixie	Use libtiff-dev instead
No module named 'RPi'	RPi.GPIO missing	pip install RPi.GPIO spidev
No route to host	Wrong HA IP address	Check IP in HA Settings > Network
Status 400 return_response	Missing URL parameter	Append ?return_response to URL
Display stays white	Wrong VCOM value	Read VCOM from FPC ribbon cable
No autostart after reboot	Service not enabled	systemctl enable epaper.service

Where to Find Key Values

• VCOM value: printed on the FPC ribbon cable of the display

• HA IP: Home Assistant → Settings → System → Network

• HA Token: Home Assistant → Profile → Security → Long-Lived Access Tokens

• Entity IDs: Home Assistant → Settings → Devices & Services → Entities

---

Raspberry Pi 2B | Raspberry Pi OS Trixie (Debian 13) | Waveshare 6" HD e-Paper HAT IT8951